From Actions to Understanding: Conformal Interpretability of Temporal Concepts in LLM Agents

Trilok Padhi* [1] **Ramneet Kaur*** [2], **Krishiv Agarwal** [4], **Adam D. Cobb**[2], **Daniel Elenius**[2], **Manoj Acharya**[2], **Colin Samplawski**[2], **Alexander M. Berenbeim**[3], **Nathaniel D. Bastian**[3], **Susmit Jha**[2] **Anirban Roy**[2]
*These authors contributed equally
[1]Georgia State University, Atlanta, USA
[2]Computer Science Lab, SRI, Menlo Park, USA
[3]Army Cyber Institute, United States Military Academy, West Point, NY USA
[4]Computer Science Department, University of Florida, USA

Correspondence: tpadhi1@student.gsu.edu, ramneet.kaur@sri.com

Abstract

Large Language Models (LLMs) are increasingly deployed as autonomous agents capable of reasoning, planning, and acting within interactive environments. Despite their growing capability to perform multi-step reasoning and decision-making tasks, internal mechanisms guiding their sequential behavior remain opaque. This paper presents a framework for interpreting the temporal evolution of concepts in LLM agents through a step-wise conformal lens. We introduce the conformal interpretability framework for temporal tasks, which combines step-wise reward modeling with conformal prediction to statistically label model's internal representation at each step as successful or failing. Linear probes are then trained on these representations to identify directions of temporal concepts—latent directions in the model's activation space that correspond to consistent notions of success, failure or reasoning drift. Experimental results on two simulated interactive environments, namely ScienceWorld and AlfWorld, demonstrate that these temporal concepts are linearly separable, revealing interpretable structures aligned with task success. We further show preliminary results on improving an LLM agent's performance by leveraging the proposed framework for steering the identified successful directions inside the model. The proposed approach, thus, offers a principled method for early failure detection as well as intervention in LLM-based agents, paving the path towards trustworthy autonomous language models in complex interactive settings.

1. Introduction

Large Language Models (LLMs) have rapidly evolved from static text generators [1, 33] to autonomous agents capable of navigating, planning, and acting in the physical world [16, 31, 36]. When deployed in embodied simulators such as Alfworld [29], and ScienceWorld [38], these agents exhibit impressive decision-making abilities on sequential goals for accomplishing the assigned task [21]. However, the reasoning process behind such sequential behavior remains largely opaque as these LLM agents continue to operate as *black box*, leaving practitioners without reliable explanations for *why* a model succeeds or *where* it fails within the task trajectory. This has led to increasing interest in the *interpretability* of LLMs, aimed at uncovering how models process inputs to generate outputs in a manner that is transparent and understandable to humans.

Traditional interpretability frameworks such as attribute or feature visualization [23, 40], mechanistic interpretability via sparse autoencoders [9], activation space analysis via representation engineering [46] and universal steering [6] offer valuable insights into mappings between *standalone input* and the model's latent space. These approaches are, however, fundamentally limited in capturing *temporal dynamics* within LLM-based agents as they operate on single standalone inputs such as a image, text or an image-text pair. While performing a sequential task in an interactive environment, the agent's decision at any step depends not only on the current observation but also on the entire trajectory history with prior reasoning traces, environment responses, and accumulated context. Model's internal representation space evolves through time, encoding both correct and in-

Distribution Statement "A" (Approved for Public Release, Distribution Unlimited).

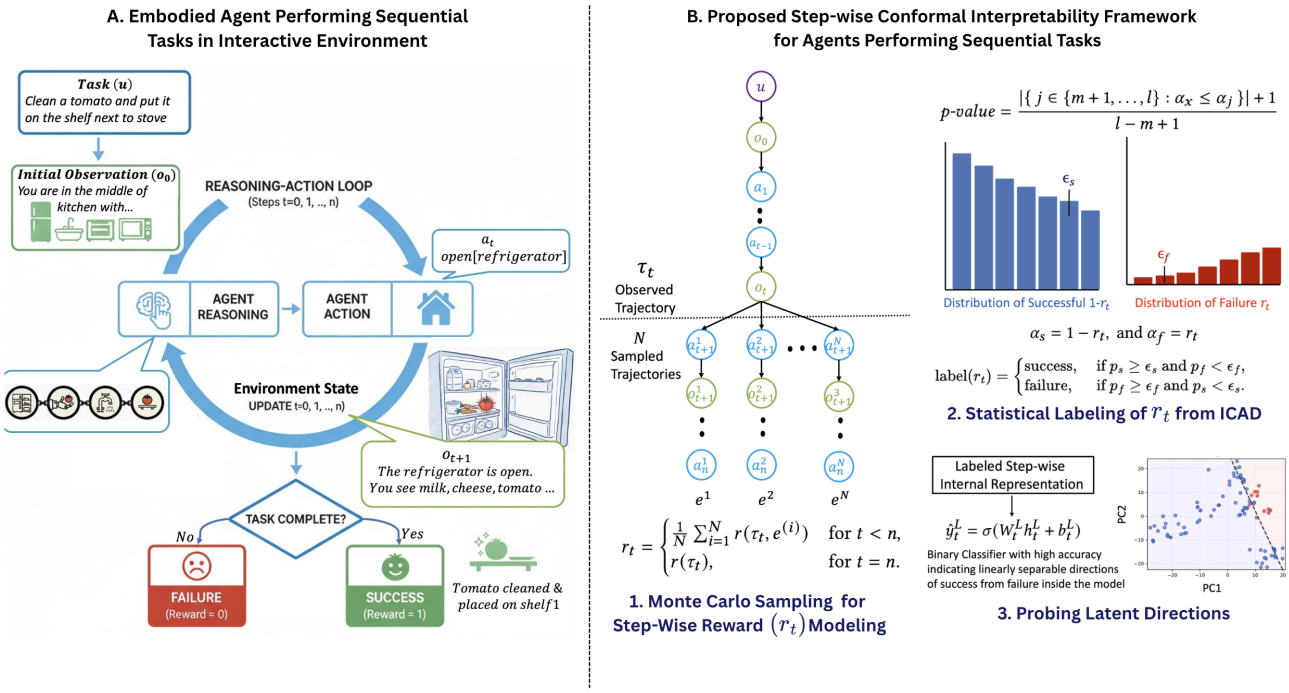


Figure 1. **A.** We consider the problem of temporal interpretability of LLM agents trained to perform sequential tasks in complex environments. **B.** Proposed framework combines step-wise reward modeling with conformal labeling to distinguish success and failure at each timestep. Linear probes are trained on the model's internal representations to test the hypothesis that step-wise notions of success and failure are linearly separable within the representation space.

correct reasoning directions. Understanding these evolving representations requires a temporally aware interpretability framework interpretability.

A straightforward approach to adapting the existing interpretability frameworks for temporal dynamics would be treating the entire trajectory as a single input and performing analysis on those. This approach, however, does not provide any granularity on *which intermediate step* of the agent contributed to success or failure of the assigned task. An agent that successfully completes the task after ten steps hides the possibility that six of those might be suboptimal or irrelevant. Without a step-level notion of "success", it becomes impossible to pinpoint when the model's internal dynamics shift from aligned reasoning to failure modes such as hallucination or irrelevant exploration.

A motivating example of the Llama-2 [33] agent trained to perform household tasks in the AlfWorld environment is as follows. For the assigned task of *"placing a roll of paper next to the toilet"*, we observed that the agent starts on the right track by moving towards the toilet and searching shelves to locate the roll. Yet midway through the trajectory, its internal reasoning begins to drift. Instead of continuing the search, the agent hallucinates that a non-existent drawer contains the paper roll, issuing an invalid action sequence. This behavioral deviation—despite earlier correct reasoning—leads to task failure, even though the initial steps were coherent and purposeful.

Such examples (more shown in Fig. 2) highlight a central challenge in *temporal interpretability*: the final success or failure of a task often obscures *where* in the sequence the agent's internal representations began to diverge. A trajectory that ultimately fails may contain multiple locally successful steps, while an apparently successful run may involve accidental recoveries after erroneous decisions. Current interpretability methods—focused on isolated prompts, static embeddings, or single-step explanations—cannot capture these evolving internal transitions, nor can they identify *when* the model's reasoning begins to deviate.

This work introduces a *step-wise conformal interpretability framework* that transforms the problem of agent's final evaluation on a multi-step sequential task into *temporal representation analysis*. We posit that the *directions of success and failure are geometrically separable* in an LLM fine-tuned to perform sequential tasks, .i.e, these models internally have separable notions of success and failure. To test this hypothesis, we build the proposed interpretability framework on the following three key components (Fig.1(B)):

1. Step-Wise Reward Modeling: We follow Xiong et al. [41]'s approach on generating fine-grained step-wise rewards using Monte Carlo sampling over future trajectories.

This transforms sparse final rewards into dense temporal feedback signals on success (or failure) of every step.

2. Statistical Labeling via Conformal Prediction: We propose using Inductive Conformal Prediction (ICP) to label model's internal representation at each step as successful or failing with provable confidence bounds.

3. Probing Latent Directions: We train classifiers (or linear probes) on layer and time-conditioned agent's activations to distinguish the latent space of success from failure.

Key Findings. High accuracy and F1-scores of linear probes on Llama2-7B in two complex simulated environments, namely ScienceWorld [38] and AlfWorld [29], validate our hypothesis that an LLM fine-tuned for sequential tasks develops an internal notion of linearly separable stepwise success across (a) timesteps, (b) layers, and (c) domains: both in-distribution and out-of-distribution.

Steering the model towards Success. As observed in prior work [28, 39] and validated in our experiments, these agents lack the intrinsic ability to self-correct themselves back towards the success directions and often continue drifting along failure trajectories once the deviation occurs. The proposed framework lays the foundation for *steering* LLM agents towards the right direction by enabling targeted interventions when early signs of hallucination or misalignment emerge in their internal representations. We conduct preliminary experiments on steering the model towards its (identified) internal 'success' directions and observe that the accuracy of the steered model improves from the baseline model, providing evidence on the practical use of the proposed framework.

2. Related Work

LLM Agents for Interactive Embodied Environments. LLMs have evolved beyond text generation to function as powerful policy models for decision-making in interactive environments. Early systems like WebGPT [26] and SimpleTOD [13] used human feedback or annotated dialogues for interactive learning, while ReAct [43] demonstrated a more scalable approach by integrating natural language reasoning directly into the decision loop, removing human from the loop. Building on this paradigm, works such as IPR [41], SayCan [2] and Inner Monologue [15] extend LLM-based decision-making to embodied and interactive domains, where agents reason over both language and environmental feedback. We perform interpretability analysis on LLMs fine-tuned using ReACT style to autonomously perform sequential tasks in interactive environments.

Interpretability of Large Language Models (LLMs). Existing interpretability methods include attribution techniques (e.g., saliency maps and masking-based perturbations) that relate outputs to input tokens [5, 20], and post-hoc methods where the model explains or verbalizes its reasoning [14, 24]. These approaches, while broadly applicable, provide limited insight into the mechanisms underlying model behavior. Mechanistic interpretability instead analyzes internal activations, either at the level of individual neurons or distributed features [6, 9, 46]. However, most work is static, associating standalone inputs with concepts and ignoring how such concepts evolve over time as models reason and act. Time-series interpretability for LLMs has been explored primarily in domains such as weather, finance, and health [37, 45], but not in the interactive agentic settings we study.

Use of Probes for Interpretability Analysis. Research on probes in LLMs has developed into a major branch of linguistic and interpretability analysis. Early studies [8, 12], used linear probes to show that intermediate layers of models like BERT [32] encode rich linguistic structures, including parts of speech, syntactic trees, and semantic roles. Foundational work by Alain and Bengio [3] introduced the concept of auxiliary classifiers as probes, inspiring subsequent analysis of internal representations in neural networks. Recent work shows that truthful answers to factual questions correspond to approximately linear directions in activation space [25]. We similarly use linear probes on hidden states of an LLM agent to test whether success vs. failure during sequential tasks is encoded along separable directions over time, using probe accuracy as a proxy for this separability.

Conformal Prediction in Explainable AI. Conformal prediction (CP) provides distribution-free uncertainty quantification via calibrated prediction sets with guaranteed coverage [4]. Recent work has begun to apply CP to LLMs for uncertainty estimation, out-of-distribution detection, safety, and evaluation [10, 11, 18, 27, 34**?**]. In contrast, we use CP in a novel way: to assign calibrated success/failure labels to temporal representations of an LLM agent, yielding bounded error rates when constructing trajectory-level interpretability signals.

3. Background

3.1. Performing Sequential Tasks in Interactive Environment

Problem Setting: As shown in Fig. 1 (A), we consider a general class of sequential decision-making tasks for LLM agents in an interactive environment. Specifically, an LLM interacts with the environment by receiving textual observations in response to each action performed by the model. The goal of the agent is to execute a coherent sequence of steps that lead to the successful task completion while reasoning and acting at each step through natural language.

Task Formulation: Formally, the task can be represented as a partially observable Markov decision process, described by the tuple (U, S, O, A, T, R) where U represents the space of natural language task instructions, S is the set

of environment's states and O is the corresponding observation space providing textual information on the environment's state, and A denotes the discrete action space defined in natural language. The transition dynamics $T : S \times A \rightarrow S$ govern how the environment's state evolves after an action, and the reward R provides scalar feedback indicating the overall task performance of the agent.

Task Execution: At the start of each episode, the agent is provided with a task instruction $u \in U$ and an initial observation o_0 that describes the environment's initial state s_0. At any discrete time step t, the agent takes an action according to its learned policy π_θ:

$$a_t \sim \pi_\theta(\cdot \mid \tau_{t-1}).$$

Here $\tau_{t-1} = (u, s_0, a_0, \ldots, a_{t-2}, s_{t-1})$ is the episode's history till the previous time-step. Upon executing a_t, the environment transitions to a new state $s_t = T(s_{t-1}, a_t)$ and produces the corresponding observation o_t for the LLM. This iterative process continues until the task is successfully completed or the maximum number of steps is reached. At the end of episode, the environment provides a scalar reward summarizing the overall task performance of the agent.

Example Task for a Household LLM Agent

Task: $u =$ *"Clean a tomato and put it on the shelf next to the stove."*

Initial Observation: $o_0 =$ *"You are in the middle of a kitchen. You observe a refrigerator, washbasin, microwave, shelf1 next to stove, and shelf2 next to basin."*

To complete the task, the agent samples a sequence of subtasks from its learned policy, such as opening the refrigerator to find a tomato, washing it in the washbasin, and placing it on shelf1 next to the stove. After each action, the environment returns a new textual observation.

For example, after action a_t = *open[refrigerator]*, the environment might return: $o_{t+1} =$ *"The refrigerator is now open. You see milk, tomato, cheese, and carrots."*

This iterative action–observation loop continues until the task is completed or the step limit is reached, with the correctness of each action contributing to the overall success. At the end of the episode, the environment provides a scalar reward summarizing the agent's task performance, e.g., a value of 1 if the tomato is washed and correctly placed.

3.2. Supervised Fine-Tuning of LLM Agents

To equip a large language model (LLM) with core agentic capabilities, we perform *supervised fine-tuning* (SFT) on expert demonstrations, aligning the model's policy with trajectories that exemplify correct reasoning, decision-making, and action execution in interactive environments [41].

Expert Demonstrations and the ReAct Paradigm: In agentic settings, the model must not only produce the correct action but also reason about why that action is appropriate given the current context. To capture this reasoning–action interplay, we adopt the *ReAct* (Reasoning and Acting) format [43] of the expert demonstrations for SFT. Here, each step taken by the agent consists of a natural language reasoning trace followed by an executable action. An example on how the ReAct step looks like is as follows: `Thought: I need to open the refrigerator to check for tomato. Action: open[refrigerator].`

Such paired data explicitly teach the model how to alternate between reflective reasoning (`Thought`) and executable (`Action`) in the environment, promoting explainability in its downstream behavior.

Expert Trajectory Dataset Construction: Let $ET = \{(u^{(i)}, \tau^{(i)})\}_{i=1}^{|ET|}$ denote a collection of *E*xpert *T*rajectories, where $u^{(i)}$ is the natural language task instruction and $\tau^{(i)} = (s_o, a_0, s_1, \ldots, a_{final}, s_{final})$ is the sequence of actions and corresponding states (expressed as observations in natural language) from the expert's interaction with the environment. Each trajectory is annotated in ReAct form, providing the reasoning text before each action. These demonstrations can originate from human experts, high-performing teacher models such as GPT [1], or curated datasets collected through scripted interaction with simulation environments.

Training Objective: During SFT, the LLM learns to imitate the expert's behavior by maximizing the likelihood of the expert trajectory given the task instruction:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(u,\tau)\sim ET}\big[\log \pi_\theta(\tau \mid u)\big],$$

Here, θ is the set of model parameters. In practice, the joint probability of the entire trajectory can be decomposed into a sequence of conditional action probabilities:

$$\pi_\theta(\tau \mid u) = \prod_{t=1}^{final} \pi_\theta(a_t \mid u, s_o, a_0, \ldots, s_{t-1}),$$

which leads to a token-level autoregressive training objective:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(u,\tau)\sim ET}\Big[\sum_{t=1}^{final} \log \pi_\theta(a_t \mid u, \tau_{t-1})\Big],$$

where τ_{t-1} denotes the trajectory history up to step $t-1$. This objective encourages the agent to reproduce the expert's next action at each step, conditioned on the task and its full prior context on its previous steps and the environment's intermediate observations.

3.3. Conformal Prediction

Conformal prediction [4] is a statistical framework for quantifying how well a new input sample aligns with a ref-

erence data distribution. At its core, the framework relies on a *non-conformity measure* (NCM), a real-valued function that quantifies the extent to which an input deviates from the behavior observed in the reference data. Given a dataset $X = \{x_1, x_2, \ldots, x_l\}$ drawn i.i.d. from an underlying data distribution $\mathcal{D}$ of interest, a non-conformity score α_x is assigned to the input x by the NCM defined on $X \cup \{x\}$. Larger value of α_x indicates greater deviation from $\mathcal{D}$ and, consequently, a higher likelihood that the sample is atypical.

Classical Conformal Anomaly Detection: Conformal Anomaly Detection (CAD) utilizes the non-conformity score to assess the likelihood that an unseen input belonging to the same distribution as $\mathcal{D}$. This is done by computing p-value of the input x by comparing its non-conformity score α_x with those of the datapoints in X from the NCM defined on the new set $X \cup \{x\}$:

$$p\text{-}value = \frac{|\{\, i \in \{1, \ldots, l\} : \alpha_x \leq \alpha_i \,\}| + 1}{l + 1}. \quad (1)$$

If x is sampled from $\mathcal{D}$, its p-value will tend to be large; conversely, inputs exhibiting high deviation from $\mathcal{D}$ yield smaller p-values. An input is deemed anomalous when $p\text{-}value < \epsilon$, where $\epsilon \in (0, 1)$ is a user-defined significance level controlling the allowable false-alarm probability.

Inductive Conformal Anomaly Detection [35]: While the classical formulation in (1) is statistically sound, recomputing NCM scores across the entire set X for every test input is computationally expensive. The *Inductive Conformal Anomaly Detection* (ICAD) framework mitigates this cost by partitioning the data into a proper training subset $X_{\text{tr}} = \{x_1, \ldots, x_m\}$ and a calibration subset $X_{\text{cal}} = \{x_{m+1}, \ldots, x_l\}$. The NCM is defined on X_{tr} and then evaluated for each calibration datapoint to produce the set of calibration scores $\{\alpha_j\}_{j=m+1}^{l}$. For a new input x, its non-conformity is assessed by comparing its score α_x against the calibration scores:

$$p\text{-}value = \frac{|\{\, j \in \{m+1, \ldots, l\} : \alpha_x \leq \alpha_j \,\}| + 1}{l - m + 1}, \quad (2)$$

and, again, detecting x as anomalous if the computed $p\text{-}value < \epsilon$. This inductive formulation permits all calibration scores to be pre-computed offline, thereby enabling efficient inference while retaining the theoretical validity of the conformal framework.

Statistical Validity and Error Control [35]: Under the standard i.i.d. assumption that both calibration and test samples are drawn from the same distribution $\mathcal{D}$, the p-values obtained via (2) are uniformly distributed in the interval $(0, 1)$. Consequently, the probability of a false alarm—incorrectly identifying an in-distribution sample as anomalous—is provably bounded by the significance threshold ϵ:

$$\Pr[p(x \in \mathcal{D}) < \epsilon] \leq \epsilon. \quad (3)$$

The efficacy of the conformal framework depends on the underlying NCM. A variety of NCMs have been proposed in literature, employing methods such as k-nearest neighbors [35], variational autoencoders [7], memory prototypes [42], and transformation equivariance [17, 19]. We propose using the step-wise reward to define the NCM for labeling each step as sampled from the distribution of successful or failing step.

4. Identifying Directions for Temporal Concepts in LLM Agents

For an LLM agent trained to perform a multi-step sequential task, we hypothesize that the directions of success and failure for the task become linearly separable in the agent's internal representation space across layers and time. We aim to validate this hypothesis by training linear probes to classify the agent's activation space as success or failure at each step of the task.

The notion of success (or failure) is quantified by the *step-wise reward* assigned to the agent at each sequential step taken to accomplish the task. Here, we provide details on a) generating these step-wise rewards via Monte-Carlo sampling of agent's trajectory from its learned policy on performing these tasks, b) leveraging inductive conformal anomaly detection (ICAD) framework for labeling these step-wise rewards as success (or failure) with bounded probability on making labeling errors, and c) training linear probes to classify the agent's internal activation space at each step as success or failure from the labeled step-wise reward.

4.1. Generating Step-Wise Rewards

Traditional learning based approaches rely solely on the *final reward* r assigned to the agent's complete trajectory on its ability to complete the task. This obscures the contribution of individual steps towards the goal. To achieve granular interpretability, we propose to use *step-wise rewards* $\{r_t\}_{t=1}^{T}$, where r_t represents the quantitative measure of success for the agent's partial trajectory τ_t till time t:

$$\tau_t = (s_0, a_0, s_1, a_1, \ldots, s_{t-1}, a_{t-1}, s_t).$$

To estimate r_t, we follow Xiong et al. [41]'s approach on performing Monte Carlo sampling on the agent's action space from its learned policy π_θ conditioned on the observed trajectory. Specifically, given τ_t, we generate N complete, subsequent expected trajectories

$$e^{(i)} = (a^i_{t+1}, s^i_{t+1}, \ldots a^i_{final}, s^i_{final}),$$

by performing iterative Monte Carlo sampling on the agents action space starting from a_t till a^i_{final}.

The step-level reward r_t at time t is calculated as:

$$r_t = \begin{cases} \frac{1}{N} \sum_{i=1}^{N} r(\tau_t, e^{(i)}) & \text{for } t < n, \\ r(\tau_t), & \text{for } t = n. \end{cases}$$

This procedure allows the model to evaluate the *expected future success probability* measured in terms of step-wise rewards that are conditioned on its current state. The use of Monte Carlo sampling transforms interpretability into a probabilistic, forward-looking signal, better aligning with how agents internally (intends to) plan the task execution.

4.2. Labeling Step-Wise Rewards

Once step-wise rewards are estimated, the key challenge is determining *when a reward should be interpreted as success or failure.* We introduce the **Conformal Prediction–based labeling mechanism** to statistically calibrate both success and failure labels with bounded guarantees on making the labeling error.

Given the calibration set of step-wise rewards for both successful and failure steps, we can calculate the (non-)conformity of r_t w.r.t success as well as failure. This gives us two p-values for the step: one corresponding to the likelihood of it being a successful step (p_s), and the other corresponding to the likelihood of it being a failure step (p_f). With the intuition of higher rewards for successful steps than failure, we propose

$$\alpha_s = 1 - r_t, \text{ and } \alpha_f = r_t.$$

as the non-conformity score with respect to the successful and failure steps respectively. r_t is then labeled as:

$$\text{label}(r_t) = \begin{cases} \text{success}, & \text{if } p_s \geq \epsilon_s \text{ and } p_f < \epsilon_f, \\ \text{failure}, & \text{if } p_f \geq \epsilon_f \text{ and } p_s < \epsilon_s. \end{cases} \quad (4)$$

Theorem 1 (Bounded Guarantees on making Labeling Errors)**.** *The probability of labeling a successful step-wise reward as failure (or False Negative Rate) is strictly bounded by ϵ_s, and the probability of labeling a failure step-wise reward as success (or False Positive Rate) is strictly bounded by ϵ_f.*

Proof. The proof without strict bounds, i.e. without the 'and' (intersection) conditions in (4):

$$\text{label}(r_t) = \begin{cases} \text{success}, & \text{if } p_s \geq \epsilon_s, \\ \text{failure}, & \text{if } p_f \geq \epsilon_f. \end{cases}$$

follows directly from the statistical guarantees of the inductive conformal anomaly detection (ICAD) framework (3). In other words, the probability of p-value for successful r_t less than ϵ_s is bounded by ϵ_s. The second condition $p_f < \epsilon_f$ is an intersection, which reduces the overall probability. Therefore, the false negative rate is strictly bounded by ϵ_s. Similarly, the probability of p-value for failure r_t less than ϵ_f is bounded by ϵ_f. The second condition $p_s < \epsilon_s$ is an intersection, which reduces the overall probability. Therefore, the false positive rate is strictly bounded by ϵ_f. □

4.3. Representation Probing Across Timesteps

At each time step t, the model's hidden representation captures contextualized knowledge of its decisions till time t. We define the internal state $\mathbf{h}_t^L$ of an LLM agent as its residual stream activations at a specific layer L and at the last token position corresponding to the trajectory τ_t till time t. This state $\mathbf{h}_t^L$ is the object of our interpretability study.

The challenge is to map these latent representations to an interpretable success/failure signal at every timestep. Having obtained calibrated step-wise reward labels, we investigate whether the corresponding model's hidden representations for success and failure are linearly separable. For this, we then train **linear probes** P_t^L to classify h_t^L as success vs. failure: $\hat{y}_t^L = \sigma(W_t^L h_t^L + b_t^L)$, where W_t^L and b_t^L are the (weight and bias) probe parameters.

Classification accuracy of probe provides a quantitative measure of how distinctly the model encodes success and failure trajectories within its internal representation space [3, 25].

5. Experimental Results

5.1. Case Study I: ScienceWorld

Our first case study is on sequential tasks in ScienceWorld [38], a large-scale text-based environment for evaluating an LLM agent's ability to perform scientific reasoning and procedural tasks.

Environment: Each instance of the environment represents a small virtual world inspired by elementary science domains—such as physics, chemistry, and biology—where the agent must explore, manipulate, and reason about objects to accomplish experiment-style goals. The environment comprises interconnected rooms (e.g., greenhouse, laboratory, workshop), each populated with interactive objects that support diverse affordances. Agents operate entirely through natural-language commands and receive textual feedback describing environmental state changes. Fig. 3 in the Appendix illustrates a representative task, "Testing Conductivity", along with the corresponding action-observation trajectory. Tasks in ScienceWorld are typically long-horizon and a normalized reward in $[0, 1]$, which is assigned at the end of each episode to reflect the agent's overall task performance. Specifically, each task is decomposed into multiple sub-goals, and the final reward is computed based on how many of these sub-goals the agent achieves, thereby enabling a fine-grained evaluation of procedural and scientific reasoning capabilities. Examples of these sub-goals are illustrated in Fig. 2, where the preferred sequence of steps is mentioned in the task description.

LLM Agent: We train the `Llama-2-7B` model [33] on 60% (889 out of 1443) of the training trajectories. The agent is trained using Supervised Fine-Tuning (SFT) on a

curated dataset of expert trajectories formatted in the ReAct paradigm [43], which explicitly interleaves natural language `Thought` and executable `Action` steps. The remaining 40% is split equally between the calibration set for conformal labeling of step-wise rewards and the training set of probes on residual stream activations of all 32 layers on the last token of the entire trajectory till time t. We observe that the trained agent is mostly able to successfully complete the assigned task within 10 timesteps.

Testing Scenarios: The environment offers two types of test scenarios: seen and unseen. Seen test set comprises of those tasks (or variations of tasks) that the agent encountered and learned from during its training phase. We refer to this test set as *in-distribution* because it falls within the scope of the data the model was exposed to. Unseen test set comprises of those tasks that the agent never encountered during its training. These involve novel combinations of objects, new environmental layouts, or even entirely new scientific concepts. We refer to this test set as *out-of-distribution* (OOD). We test on the entire set of 360 in-distribution and 165 OOD tasks.

Conformal Thresholds: We set $\epsilon_s = \epsilon_f = 0.1$ for labeling step-wise rewards as success or failure all timesteps. This strictly bounds both the false negative and false positive labeling errors to 10% [17].

Results and Observations: Tables 1 and 2 show accuracy on iD and OOD test set respectively from timesteps $t = 2$ to 10. At timestep 1, the model is given instructions on its role in the ScienceWorld environment and it always (irrespective of success or failure) responds with an 'OK'. Probes achieve significantly high accuracy with upto 100% in most test cases for iD set and good accuracy in most test cases for OOD set except for one test case (50% at $t = 10$ for layer 8).

Tables 3 and 4 show F1 scores on the in-distribution and OOD test sets, respectively. Similar to results on accuracy, F1 scores are also high on most test cases of the iD set except for one test case (0.67 at $t = 10$ for layer 8). These scores are also high on most test cases of the OOD set except for three cases: 0.67 at $t = 3$ for layers 24 and 32 and for layer 8 at $t = 8$. We observe similar results across all layers of the model, and selected early (layer 8), middle (16 and 24) and later (32) layers to show the results. These results validate our hypothesis that directions of success and failure become separable in an LLM fine-tuned to perform sequential tasks.

Layer	t=2	t=3	t=4	t=5	t=6	t=7	t=8	t=9	t=10
8	100	100	100	100	100	83	91	91	100
16	100	100	100	100	100	92	100	91	100
24	100	93	100	100	100	92	100	91	100
32	100	93	100	100	100	92	91	91	100

Table 1. Accuracy(%) of Linear Probes on in-distribution Test Set of ScienceWorld across Layers and Timesteps.

Layer	t=2	t=3	t=4	t=5	t=6	t=7	t=8	t=9	t=10
8	100	92	100	94	92	80	75	75	50
16	100	92	93	94	92	80	75	75	100
24	100	92	100	94	92	73	75	75	100
32	100	92	100	94	92	73	75	75	100

Table 2. Accuracy(%) of Linear Probes on OOD Test Set of ScienceWorld across Layers and Timesteps.

Layer	t=2	t=3	t=4	t=5	t=6	t=7	t=8	t=9	t=10
8	1.00	0.95	1.00	0.97	0.96	0.88	0.86	0.80	0.67
16	1.00	0.95	0.96	0.97	0.96	0.88	0.86	0.80	1.00
24	1.00	0.95	1.00	0.97	0.96	0.83	0.86	0.80	1.00
32	1.00	0.95	1.00	0.97	0.96	0.83	0.86	0.80	1.00

Table 3. F1 Score of Linear Probes on in-distribution Test Set of ScienceWorld across Layers and Timesteps.

Layer	t=2	t=3	t=4	t=5	t=6	t=7	t=8	t=9	t=10
8	1.00	1.00	1.00	1.00	1.00	1.00	0.67	0.86	0.80
16	1.00	1.00	1.00	1.00	1.00	0.80	1.00	0.80	1.00
24	1.00	0.67	1.00	1.00	1.00	0.80	1.00	0.80	1.00
32	1.00	0.67	1.00	1.00	1.00	0.80	0.86	0.80	1.00

Table 4. F1 Score of Linear Probes on OOD Test Set across of ScienceWorld Layers and Timesteps.

5.2. Case Study II: AlfWorld

The second case study evaluates our conformal interpretability framework on the *ALFWorld* environment [29]. AlfWorld serves as a widely adopted benchmark for autonomous LLM agents in interactive embodied settings, requiring complex sequential reasoning and navigation in simulated household environment [22, 44].

Environment: ALFWorld is a text-based environment that grounds language instructions in a simulated household world derived from ALFRED [30]. Tasks involve multi-step goals such as fetching, cleaning, heating, or putting away objects (e.g., "put a clean mug in the cabinet"), requiring sequential decision-making, long-horizon planning, and interaction with diverse objects and room states. The agent receives textual observations and issues text-based actions (e.g., `go to kitchen`, `pick up mug`); performance is evaluated by whether the final goal is successfully completed. Fig. 3 (Appendix) shows an example trajectory for a task of cleaning a tomato and placing it on a shelf next to the stove.

LLM Agent and Training: Similar to ScienceWorld, here also we train Llama-2-7B [33] on 60% (1710 out of 2851) of the training trajectories and split the remaining equally between calibration set and training the probes on

all layers and timesteps.

Conformal Thresholds: Again, we set $\epsilon_s = \epsilon_f = 0.1$ for all timesteps $t \in \{1, 2, \ldots, 10\}$, strcitly bounding both false negative and false positive labeling errors to 10%.

Results and Discussion: Tables 5 and 6 show the accuracy and F1 scores of the trained probes on the test set of the AlfWorld, respectively. The accuracy varies from 60% (layer 8 at $t = 6$ and layer 24 at $t = 4$) to 95% (at $t = 2$), and the F1 score varies from 0.56 (layer 24 at $t = 4$ to 0.95 (at $t = 2$). Although these results are comparable to the probe accuracies reported for distinguishing truth and falsehood directions in factual question settings [25], they are lower than our results on ScienceWorld. We hypothesize that this difference arises because ALFWorld provides only a single final reward upon task completion, whereas ScienceWorld offers intermediate rewards for sub-goals at multiple steps. This denser reward structure in ScienceWorld yields more precise step-wise feedback, leading to improved predictive performance.

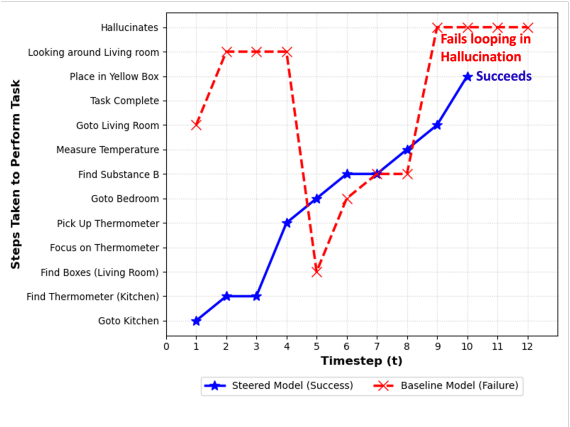
Layer	t=2	t=3	t=4	t=5	t=6	t=7	t=8	t=9	t=10
8	95	85	75	75	60	80	75	70	75
16	95	90	65	65	75	65	80	80	75
24	95	90	60	85	80	60	80	75	75
32	95	85	75	85	80	75	90	80	75

Table 5. Accuracy (%) of Linear Probes on ALFWorld Test Set across Layers and Timesteps.

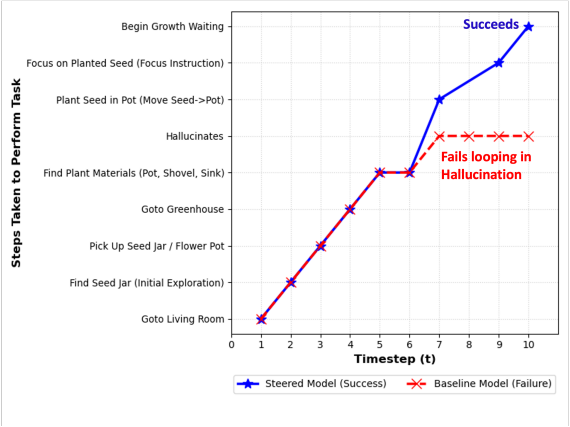
Layer	t=2	t=3	t=4	t=5	t=6	t=7	t=8	t=9	t=10
8	0.95	0.82	0.71	0.76	0.64	0.82	0.76	0.62	0.71
16	0.95	0.89	0.59	0.63	0.74	0.63	0.78	0.78	0.67
24	0.95	0.89	0.56	0.84	0.80	0.60	0.80	0.71	0.67
32	0.95	0.82	0.74	0.86	0.80	0.74	0.89	0.78	0.67

Table 6. F1 Score of Linear Probes on ALFWorld Test Set across Layers and Timesteps.

Steerability of Models Towards Success: Using the labeled internal representations provided by our framework, we apply the existing steering approach Representation Engineering (RepE) [46] to compute contrastive activation directions between successful and failing trajectories, and add these directions during inference to steer the agent's internal states toward success while preserving overall performance. Motivated by the empirical observation that success and failure trajectories begin to diverge at timestep 3, we perform a single early intervention at step 3 on a supervised fine-tuned Llama-2-7B agent in ScienceWorld. We use a small steering coefficient of 0.025 as selected by following RepE's procedure. This intervention yields a 1.1% accuracy boost, which is competitive with substantially more expensive methods—Best-of-N sampling, Rejection Sampling Fine-Tuning (RFT), and Direct Preference Optimization (DPO)—that obtain 2.8%, 4.2%, and 6.8% accuracy gains, respectively, on the same test settings.



(a) Task: "Measure the temperature of substance B in bedroom. First, focus on the thermometer. Next, focus on the unknown substance B. Then, if temperature $> 0.0°C \rightarrow$ place it in the yellow box; if $< 0.0°C \rightarrow$ place it in the purple box. Boxes are in the living room." The steered model (blue) prioritizes the correct instruction order and completes the task, while the baseline (red) follows the wrong order and loops due to hallucinating possession of the thermometer.



(b) Task: "Grow Plant to Reproduction. Seeds can be found in the living room. First, focus on a seed. Then, modify the environment to grow the plant until it reaches the reproduction stage." The baseline model (red) initially follows the correct sequence but later drifts due to hallucination.

Figure 2. Comparison of baseline (SFT Llama-2-7B) and steered LLM agents – Steering along learned *success directions* mitigates reasoning drift, reduces hallucinations, and improves task completion.

Fig. 2 illustrates two test cases where the steered model corrects common failure modes: (a) failing to follow the correct sequence of sub-goals and (b) drifting off track mid-trajectory. The reported steering results are preliminary as we applied an off-the-shelf steering approach (RepE) only at timestep 3. RepE has been proposed to steer standalone inputs. Leveraging the proposed framework for developing and applying steering approaches for temporal data across steps is one of the future directions.

6. Conclusion

This work introduces a conformal time-series interpretability framework for analyzing the evolving internal representations of LLM agents on sequential tasks. By combining step-wise reward modeling with inductive conformal prediction, it assigns statistically calibrated success/failure labels to internal states at each timestep, enabling fine-grained temporal analysis and linear probing of success vs. failure directions. Experiments on two complex interactive environments show that these directions are linearly separable and can be used to steer agents toward more successful outcomes, supporting early detection of misalignment and step-level intervention. Future work will extend the framework to multimodal embodied settings and explore proactive steering of autonomous LLM agents via interpretable temporal feedback.

Acknowledgments

This material is based on work supported by the United States Air Force and Defense Advanced Research Projects Agency (DARPA) under Contract No. FA8750-23-C-0519, the Defense Advanced Research Projects Agency (DARPA) under Agreement No. HR0011-24-9-0424. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the Defense Advanced Research Projects Agency (DARPA).

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. 1, 4

[2] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022. 3

[3] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. In *Proceedings of the International Conference on Learning Representations (ICLR) Workshop Track*, 2017. 3, 6

[4] Vineeth Balasubramanian, Shen-Shyang Ho, and Vladimir Vovk. *Conformal prediction for reliable machine learning: theory, adaptations and applications*. Newnes, 2014. 3, 4

[5] Oren Barkan, Yonatan Toib, Yehonatan Elisha, Jonathan Weill, and Noam Koenigstein. Llm explainability via attributive masking learning. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 9522–9537, 2024. 3

[6] Daniel Beaglehole, Adityanarayanan Radhakrishnan, Enric Boix-Adsera, and Mikhail Belkin. Toward universal steering and monitoring of ai models. *arXiv preprint arXiv:2502.03708*, 2025. 1, 3

[7] Feiyang Cai and Xenofon Koutsoukos. Real-time out-of-distribution detection in learning-enabled cyber-physical systems. In *2020 ACM/IEEE 11th International Conference on Cyber-Physical Systems (ICCPS)*, pages 174–183. IEEE, 2020. 5

[8] Alexis Conneau, Germán Kruszewski, Guillaume Lample, Loïc Barrault, and Marco Baroni. What you can cram into a single $&!* vector: Probing sentence embeddings for linguistic properties. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 2126–2136, 2018. 3

[9] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023. 1, 3

[10] Achref Doula, Max Mühlhäuser, and Alejandro Sanchez Guinea. Safepath: conformal prediction for safe llm-based autonomous navigation. *arXiv preprint arXiv:2505.09427*, 2025. 3

[11] Ayush Gupta, Ramneet Kaur, Anirban Roy, Adam D Cobb, Rama Chellappa, and Susmit Jha. Polysemantic dropout: Conformal ood detection for specialized llms. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 11768–11781, 2025. 3

[12] John Hewitt and Christopher D. Manning. A structural probe for finding syntax in word representations. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, pages 4129–4138, 2019. 3

[13] Ehsan Hosseini-Asl, Bryan McCann, Chien-Sheng Wu, Semih Yavuz, and Richard Socher. A simple language model for task-oriented dialogue. *Advances in Neural Information Processing Systems*, 33:20179–20191, 2020. 3

[14] Shiyuan Huang, Siddarth Mamidanna, Shreedhar Jangam, Yilun Zhou, and Leilani H Gilpin. Can large language models explain themselves? a study of llm-generated self-explanations. *arXiv preprint arXiv:2310.11207*, 2023. 3

[15] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, et al. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022. 3

[16] Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. Understanding the planning of llm agents: A survey. *CoRR*, 2024. 1

[17] Ramneet Kaur, Susmit Jha, Anirban Roy, Sangdon Park13, Edgar Dobriban, Oleg Sokolsky, and Insup Lee. idecode: In-distribution equivariance for conformal out-of-distribution detection. 2022. 5, 7

[18] Ramneet Kaur, Colin Samplawski, Adam D Cobb, Anirban Roy, Brian Matejek, Manoj Acharya, Daniel Elenius, Alexander M Berenbeim, John A Pavlik, Nathaniel D Bastian, et al. Addressing uncertainty in llms to enhance reliability in generative ai. *arXiv preprint arXiv:2411.02381*, 2024. 3

[19] Ramneet Kaur, Yahan Yang, Oleg Sokolsky, and Insup Lee. Out-of-distribution detection in dependent data for cyber-physical systems with conformal guarantees. *ACM Transactions on Cyber-Physical Systems*, 8(4):1–27, 2024. 5

[20] Fuxiao Liu. Towards understanding in-context learning with contrastive demonstrations and saliency maps. In *NeurIPS 2024 Workshop on Scientific Methods for Understanding Deep Learning*. 3

[21] Xiaotian Liu, Ali Pesaranghader, Hanze Li, Punyaphat Sukcharoenchaikul, Jaehong Kim, Tanmana Sadhu, Hyejeong Jeon, and Scott Sanner. Open-world planning via lifted regression with llm-inferred affordances for embodied agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 20881–20897, 2025. 1

[22] Yang Liu, Weixing Chen, Yongjie Bai, Xiaodan Liang, Guanbin Li, Wen Gao, and Liang Lin. Aligning cyber space with physical world: A comprehensive survey on embodied ai. *IEEE/ASME Transactions on Mechatronics*, 2025. 7

[23] Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. *Advances in neural information processing systems*, 30, 2017. 1

[24] Qing Lyu, Shreya Havaldar, Adam Stein, Li Zhang, Delip Rao, Eric Wong, Marianna Apidianaki, and Chris Callison-Burch. Faithful chain-of-thought reasoning. In *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 305–329, 2023. 3

[25] Samuel Marks and Max Tegmark. The geometry of truth: Emergent linear structure in llm representations of true/false datasets. In *Proceedings of the Conference on Language Modeling (COLM)*, 2024. arXiv preprint arXiv:2310.06824. 3, 6, 8

[26] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021. 3

[27] Huanxin Sheng, Xinyi Liu, Hangfeng He, Jieyu Zhao, and Jian Kang. Analyzing uncertainty of llm-as-a-judge: Interval evaluations with conformal prediction. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 11297–11339, 2025. 3

[28] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In *Thirty-seventh Conference on Neural Information Processing Systems*. 3

[29] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Cote, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*. 1, 3, 7

[30] Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. Alfred: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10740–10749, 2020. 7

[31] Kaustubh Sridhar, Souradeep Dutta, Dinesh Jayaraman, and Insup Lee. Ricl: Adding in-context adaptability to pre-trained vision-language-action models. In *9th Annual Conference on Robot Learning*. 1

[32] Ian Tenney, Dipanjan Das, and Ellie Pavlick. Bert rediscovers the classical nlp pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4593–4601, 2019. 3

[33] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023. 1, 2, 6, 7

[34] Harit Vishwakarma, Alan Mishler, Thomas Cook, Niccolo Dalmasso, Natraj Raman, and Sumitra Ganesh. Prune'n predict: Optimizing llm decision-making with conformal prediction. *arXiv preprint arXiv:2501.00555*, 2024. 3

[35] Vladimir Vovk, Alex Gammerman, and Glenn Shafer. *Algorithmic learning in a random world*. Springer Science & Business Media, 2005. 5

[36] Jun Wang, David Smith Sundarsingh, Jyotirmoy V Deshmukh, and Yiannis Kantaros. Conformalnl2ltl: Translating natural language instructions into temporal logic formulas with conformal correctness guarantees. 1

[37] Jiahao Wang, Mingyue Cheng, and Qi Liu. Can slow-thinking llms reason over time? empirical studies in time series forecasting. *arXiv e-prints*, pages arXiv–2505, 2025. 3

[38] Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. Scienceworld: Is your agent

smarter than a 5th grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 11279–11298, 2022. 1, 3, 6

[39] Tianyu Wu, Shizhu He, Jingping Liu, Siqi Sun, Kang Liu, Qing-Long Han, and Yang Tang. A brief overview of chatgpt: The history, status quo and potential future development. *IEEE/CAA Journal of Automatica Sinica*, 10(5):1122–1136, 2023. 3

[40] Xiaoying Xing, Chia-Wen Kuo, Li Fuxin, Yulei Niu, Fan Chen, Ming Li, Ying Wu, Longyin Wen, and Sijie Zhu. Where do large vision-language models look at when answering questions? *arXiv e-prints*, pages arXiv–2503, 2025. 1

[41] Weimin Xiong, Yifan Song, Xiutian Zhao, Wenhao Wu, Xun Wang, Ke Wang, Cheng Li, Wei Peng, and Sujian Li. Watch every step! llm agent learning via iterative step-level process refinement. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 1556–1572, 2024. 2, 3, 4, 5

[42] Yahan Yang, Ramneet Kaur, Souradeep Dutta, and Insup Lee. Memory-based distribution shift detection for learning enabled cyber-physical systems with statistical guarantees. *ACM Transactions on Cyber-Physical Systems*, 8(2): 1–28, 2024. 5

[43] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022. 3, 4, 7

[44] Guibin Zhang, Hejia Geng, Xiaohang Yu, Zhenfei Yin, Zaibin Zhang, Zelin Tan, Heng Zhou, Zhongzhi Li, Xiangyuan Xue, Yijiang Li, et al. The landscape of agentic reinforcement learning for llms: A survey. *arXiv e-prints*, pages arXiv–2509, 2025. 7

[45] Jiahui Zhou, Dan Li, Lin Li, Zhuomin Chen, Shunyu Wu, Haozheng Ye, Jian Lou, and Costas J Spanos. Enhancing llm reasoning for time series classification by tailored thinking and fused decision. *arXiv e-prints*, pages arXiv–2506, 2025. 3

[46] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, et al. Representation engineering: A top-down approach to ai transparency. *CoRR*, 2023. 1, 3, 8

7. Supplementary Material

7.1. Task Examples from ScienceWorld and AlfWorld

Figure 3 illustrates a representative task, "Testing Conductivity" in ScienceWorld, along with the corresponding action-observation trajectory, and Figure 3 shows an example trajectory of the agent-environment interaction on one of the household's task of "cleaning a tomato and putting it on the shelf next to stove".

Distribution Statement "A" (Approved for Public Release, Distribution Unlimited).

Figure 3. Task examples from ScienceWorld and ALFWorld. **(Top)** An example of a successful trajectory for the "Test Conductivity" task in `ScienceWorld`, demonstrating the sequential reasoning required by the agent to accomplish the task. **(Bottom)** An example of a successful trajectory for the "Cleaning a tomato and putting it on the shelf next to stove" task in the ALFWorld environment, demonstrating the multi-step nature of a typical sequential task in the environment requiring commonsense, navigation, and object manipulation.

ScienceWorld: Test Conductivity

Step	Agent's Action	Environment Observation (Simplified)
1	`go to tool room`	You are in the tool room. You see a light bulb and a battery.
2	`pick up light bulb`	You picked up the light bulb.
3	`pick up battery`	You picked up the battery.
4	`go to workshop`	You are in the workshop. You see a metal fork and a plastic cup.
5	`pick up metal fork`	You picked up the metal fork.
6	`use battery on light bulb`	The battery and the light bulb are now connected in a circuit.
7	`use metal fork on light bulb`	The light bulb illuminates! The metal fork is electrically conductive.

ALFWorld: Clean tomato

Step	Agent's Action	Environment Observation (Simplified)
1	`open refrigerator`	The refrigerator is now open. You see milk, tomato, cheese, and carrots.
2	`take tomato from refrigerator`	You are now holding the tomato.
3	`go to washbasin`	You are at the washbasin. You see running water.
4	`wash tomato in washbasin`	The tomato is now clean.
5	`go to shelf1 (next to stove)`	You are near the stove. You see shelf1 next to stove.
6	`put tomato on shelf1`	The tomato has been placed on shelf1 next to the stove.

Distribution Statement "A" (Approved for Public Release, Distribution Unlimited).